Software Architecture Document (SAD)

Project: Web-Based ML Model Evaluator (WBMME)

Version: 1.0

Authors: All Members

Date: 10-10-2026

Status: Final

Revision History

Version	Date	Author	Change summary
0.2	12-10-2026	All Members	Added sections 1, 2 and 5; integrated architecture and design; aligned requirement IDs and security traceability with the SRS and Test Plan

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Document Overview
3. Architecture (3.1 to 3.9)
4. Design (4.1 to 4.6)
5. Appendices

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Web-Based ML Model Evaluator (WBMME). It explains how the system is structured, why that structure was chosen, how the components interact, and how the design meets the requirements in the Software Requirements Specification (SRS) and can be verified by the Test Plan.

1.2 Scope

Covers the web application that lets a user register and log in, upload a CSV dataset, prepare the data, select and configure one of four classifiers (Logistic Regression, k-NN, Decision Tree, Random Forest), train it, view evaluation metrics, keep a history of runs, compare runs and export results as CSV, together with an admin console and the security controls around them. Out of scope: regression and clustering, other file formats, uploaded model files, deep learning, and the optional SHAP, PDF export and Celery features, which the SRS does not commit to.

1.3 Audience

Developers, QA engineers, security reviewers, course instructors and evaluators, and anyone who maintains the system after the project.

1.4 Definitions

ADR, API, CI/CD, CSRF, CSV, MVT, NF, ML, RBAC, RTM, SAD, SO, SR, SRS, STP, STRIDE, TLS. Their meanings are listed in the glossary in section 5.1.

2. Document Overview

2.1 How to use this document

Section 3 gives the architecture: goals, stakeholders, the component diagram, component descriptions, the chosen pattern, the technology stack, risks, traceability to requirements and the security architecture. Section 4 gives the design: the data model, sequence diagrams for key flows, the API, error handling and logging, the user interface and open issues. Section 5 holds the glossary, references and tools.

Suggested reading: developers start with 3.3, 3.4 and 4.3; testers with 3.8, 4.3 and 4.4; security reviewers with 3.9 and 4.4; evaluators with 3.1, 3.5 and 3.8.

2.2 Related Documents

- Software Requirements Specification (SRS) Parts A and B: functional (MLE-F), non-functional (MLE-NF), security (MLE-SR) requirements and security objectives (MLE-SO).
- Software Test Plan (STP): test levels, test cases and the requirements traceability matrix.
- Backlog and sprint plan: 11 epics, 45 user stories in two sprints, tracked as GitHub issues.
- Mini-Project Deliverables Part-1 and Part-2 from the course.

3. Architecture

3.1 Architecture Goals and Constraints

Goals (what the architecture must achieve):

- Let a student or data scientist go from a CSV file to trained, evaluated and comparable models in one web application, with no coding.
- Keep each user's datasets, models and results private and trustworthy (MLE-SO-001 to MLE-SO-003).
- Meet the measurable targets in the SRS: upload of up to 10 MB in 5 s or less, training on up to 50,000 rows in 60 s or less, pages in 3 s or less for 20 concurrent users (MLE-NF-001 to MLE-NF-003).
- Be buildable by four students in two sprints: one deployable unit, familiar tools, small components that can be developed on separate branches.
- Be testable and traceable: every component maps to SRS requirements and to test cases in the Test Plan.

Constraints (what limits the design):

Type	Constraint
Technology	Python 3.11 and Django 5 for the backend; scikit-learn, pandas and joblib for ML; SQLite for development with PostgreSQL optional.
Scope	Only CSV upload, only four classifiers (Logistic Regression, k-NN, Decision Tree, Random Forest), classification only, CSV export only. SHAP, PDF export and Celery are optional and not committed.
Limits	Upload at most 10 MB, 20 to 50,000 rows, 2 to 100 columns, 2 to 20 target classes, 20 datasets per user, 120 requests per minute per user, 2 active jobs per user (Student 1), job time-out 300 s.
Security	Passwords hashed, HTTPS (TLS 1.2+), sessions expire after 30 minutes, login blocked for 15 minutes after 5 failed attempts in 10 minutes, CSRF protection, only system-created model files are ever loaded.

Type	Constraint
Process	Two sprints, GitHub with pull requests reviewed by the repository owner, GitHub Actions for CI, no paid services.
Team	Four students with mixed Django and Git experience, so the design avoids microservices, message brokers and separate front-end build chains.

3.2 Stakeholders and Concerns

Stakeholder	Concern	How the architecture addresses it
Student	Simple screens, quick feedback, safe use of own files	Guided Web UI flow (4.5); background training with status; per-user data isolation; Student job quota of 1.
Data Scientist	Choice of models and hyperparameters, comparison of runs, exports	Model registry with validated hyperparameters; run history and comparison; CSV export (Experiment Management).
Admin	Manage users and datasets, review activity	Role-based admin console; audit log of admin actions (Auth & Access Control, Logging & Audit).
Course instructor / evaluator	Traceable requirements, clear architecture, working demo	Traceability table (3.8), diagrams (3.3, 4.1, 4.2), CI pipeline and demo-ready Docker image.
Development team	Parallel work, small components, easy debugging	Layered design with one component per epic; stable interfaces (4.3); shared error and logging rules (4.4).
QA / testers	Testable components and observable behaviour	Each layer can be tested with stubs; error codes and log events defined in 4.4.
Security reviewer	No data leakage, no code execution from uploads	STRIDE analysis and controls in 3.9; central validation component.

3.3 Component (UML) Diagram

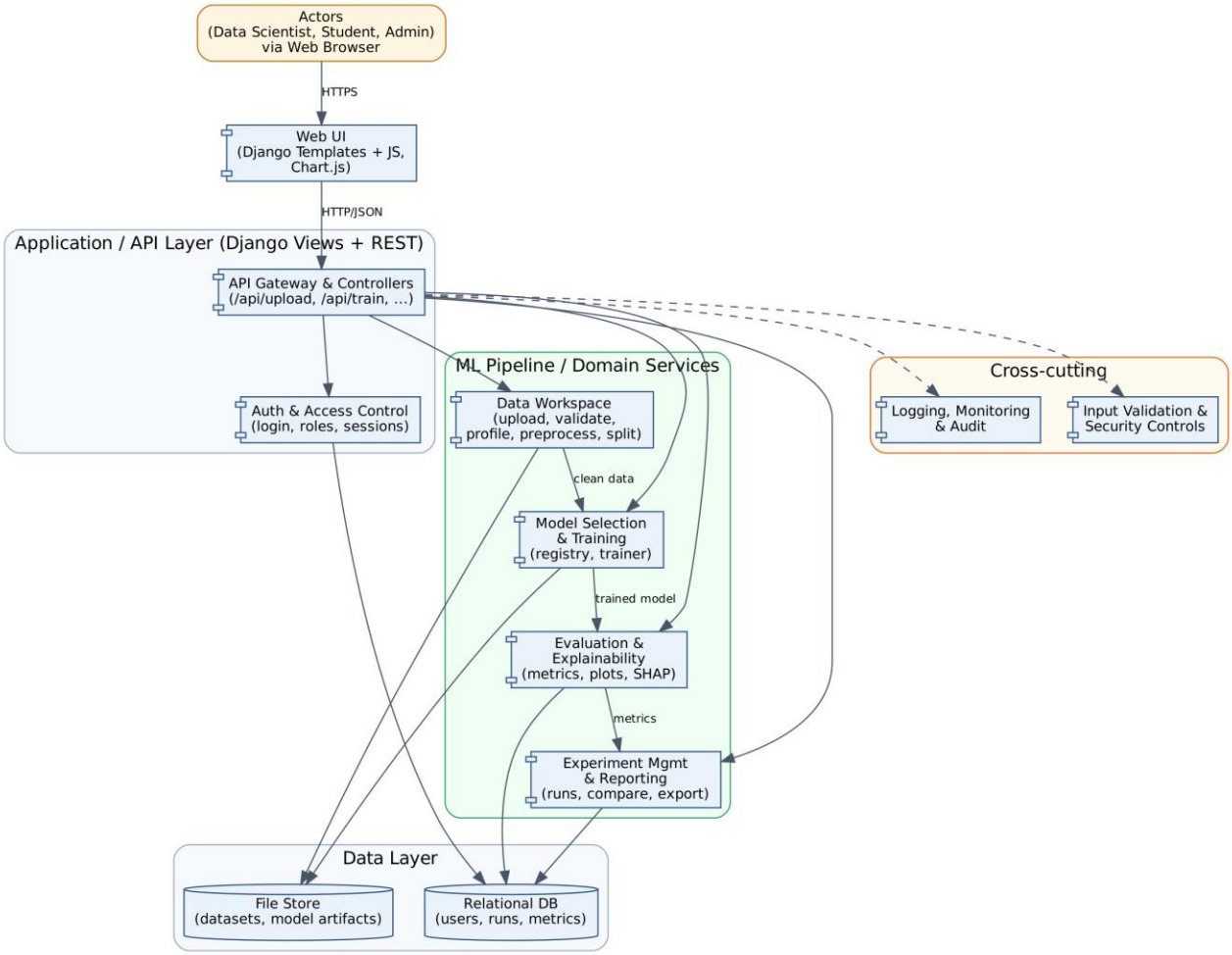


Figure 1: Component diagram. Solid arrows are runtime dependencies; dashed arrows are cross-cutting concerns applied by the API layer.

3.4 Component Descriptions

Component	Responsibility	Maps to Jira epic
Web UI	Renders pages for upload, model selection, training status, results and history. Charts (confusion matrix, metric bars) use Chart.js. Contains no business logic.	WBMME-4 to WBMME-7 (UI parts)
API Gateway & Controllers	Single entry point for all requests (/api/upload, /api/train, /api/evaluate, /api/runs). Validates input, checks authorisation, delegates to services, returns JSON.	All
Auth & Access Control	Registration, login, session handling (30 minute expiry, login throttling, CSRF protection) and role-based access (Data Scientist, Student, Admin). Ensures users only see their own datasets and runs.	WBMME-11
Data Workspace	Accepts CSV upload, validates type, size and schema, profiles the data, handles missing values and categorical encoding, and creates the train/test split.	WBMME-4
Model Selection & Training	Holds a registry of supported scikit-learn models and their hyperparameters. Trains the selected model(s) on the prepared data and stores the trained artifact.	WBMME-5
Evaluation & Explainability	Computes accuracy, precision, recall, F1, ROC-AUC (when the model gives probabilities) and the confusion	WBMME-6

Component	Responsibility	Maps to Jira epic
	matrix. Lists feature importance for models that provide it (Logistic Regression, Decision Tree, Random Forest).	
Experiment Management & Reporting	Records every run (dataset, model, parameters, metrics), supports side-by-side comparison, and exports a run or comparison as a CSV file (cells that start with = + - @ are neutralised).	WBMME-7
Logging, Monitoring & Audit	Structured logs of requests, training jobs, failures and admin actions. Never logs raw data values or credentials.	WBMME-11 (and WBMME-9 for deployment logs)
Input Validation & Security Controls	Central file validation (type, size, rows, columns), request-size limit, per-user rate limiting, job quota and time-out, and safe model serialisation.	WBMME-11
Relational DB	Stores users, roles, dataset metadata, runs and metrics.	WBMME-4 to WBMME-7
File Store	Stores uploaded datasets and trained model artifacts in per-user directories outside the web root.	WBMME-4, WBMME-5

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: Layered architecture (Presentation, Application/API, ML Pipeline/Domain, Data) with the Django MVT structure inside the web tier and a pipe-and-filter style ML pipeline (Upload, Preprocess, Train, Evaluate, Record).

Rationale:

- Separation of concerns: UI changes do not affect ML code, and a model can be swapped without touching the views.
- Matches the workflow. The ML flow is naturally sequential, so each stage is a replaceable component with a defined input and output.
- Fits the team: four students and a two-sprint schedule. A single deployable unit keeps CI/CD, debugging and the demo simple.
- Testable: each layer can be unit-tested with stubs, which supports the Test Plan's unit, integration and system levels.

Alternatives considered and rejected:

Alternative	Reason rejected
Microservices	Needs service discovery, inter-service auth and multiple deployments. Too much overhead for the team size and the sprint length.
Single-file monolith (no layers)	Fastest to start, but couples UI and ML code and makes parallel work on four branches cause merge conflicts.
Client-side ML (in-browser)	Limits model choice and dataset size, and is hard to secure and test.

3.6 Technology Stack and Data Stores

Area	Choice	Why
Language / Backend	Python 3.11, Django 5	Same language as the ML libraries; built-in auth, ORM, CSRF protection.
API style	Django views returning JSON (Django REST Framework optional)	Simple, documented endpoints for upload, train, evaluate.

Area	Choice	Why
Frontend	Django templates, JavaScript, Chart.js	No separate build chain; enough for dashboards.
ML libraries	pandas, scikit-learn, joblib (four classifiers: Logistic Regression, k-NN, Decision Tree, Random Forest)	Standard, well documented, many classic models.
Relational DB	SQLite (development), PostgreSQL (optional deployment)	Zero-setup locally; production-ready option.
File store	Local per-user directories (outside web root)	Holds datasets and model artifacts; can later move to object storage.
Background jobs	Python background worker; Celery is optional and not committed in the SRS	Keeps the UI responsive during long training.
CI/CD & quality	GitHub Actions, pytest, pytest-cov, flake8/SonarQube	Required by the guidelines; runs tests on every PR.
Packaging / Deployment	Docker (WBMME-9)	Portable, reproducible environment.
Security libs	Django auth, bcrypt/PBKDF2, HTTPS (TLS 1.2+)	Standard, audited components.

3.7 Risks and Mitigations

ID	Risk	Severity	Mitigation
R1	Long training times on large datasets block the UI or time out	High	Limit dataset size and rows; run training in a background job; show progress; set a timeout.
R2	Malicious or malformed uploads (wrong type, huge file, formula injection in CSV)	High	Whitelist .csv, check MIME type and size, parse with pandas in a safe mode, never execute content.
R3	Unsafe loading of pickled/joblib model files can run arbitrary code	High	Only load artifacts the system itself created; store them per user; never accept uploaded model files.
R4	Data leakage or overfitting gives misleading results	Medium	Fit preprocessing only on the training split; report train and test metrics; document the method.
R5	Team members unfamiliar with Git/Django cause delays and merge conflicts	Medium	Branch per feature, small PRs, reviews by repo owner, pair on first PR, shared task board.
R6	Different library versions break the build between machines	Medium	Pin versions in requirements.txt; use Docker and CI to catch drift.
R7	Users' datasets visible to other users	High	Per-user ownership checks on every query and file path; tests for access control.
R8	Sprint timeline is too short for all epics	High	Prioritise by epic order; implement a minimal end-to-end flow first; defer feature importance and CSV export if needed.

3.8 Traceability to Requirements

Req ID	Requirement (short)	Architecture component	Design reference
MLE-F-001	User registration and login	Auth & Access Control	DS-Auth-01 (4.2 login sequence)
MLE-F-002	Role-based access (Data Scientist, Student, Admin)	Auth & Access Control	DS-Auth-02

Req ID	Requirement (short)	Architecture component	Design reference
MLE-F-003	Upload CSV dataset	Data Workspace, File Store	DS-Data-01 (4.2 upload sequence, /api/upload)
MLE-F-004	Preview and profile the dataset	Data Workspace	DS-Data-02
MLE-F-005	Handle missing values and encode categories	Data Workspace	DS-Data-03
MLE-F-006	Configure train/test split	Data Workspace	DS-Data-04
MLE-F-007	Select a model from the catalogue	Model Selection & Training	DS-Model-01
MLE-F-008	Configure hyperparameters	Model Selection & Training	DS-Model-02
MLE-F-009	Train a model and show status	Model Selection & Training	DS-Model-03 (/api/train)
MLE-F-010	Compute accuracy, precision, recall, F1	Evaluation & Explainability	DS-Eval-01 (/api/evaluate)
MLE-F-011	Show confusion matrix and plots	Evaluation & Explainability, Web UI	DS-Eval-02
MLE-F-012	Feature importance / explanation	Evaluation & Explainability	DS-Eval-03
MLE-F-013	Save each run (history)	Experiment Mgmt & Reporting, DB	DS-Exp-01
MLE-F-014	Compare runs side by side	Experiment Mgmt & Reporting	DS-Exp-02
MLE-F-015	Export results report	Experiment Mgmt & Reporting	DS-Exp-03
MLE-F-016	Admin manages users and datasets	Auth & Access Control, Web UI	DS-Admin-01
MLE-NF-001	Upload of up to 10 MB processed in 5 s or less	Data Workspace	Performance budget in 3.7 R1
MLE-NF-002	Training of standard models on up to 50,000 rows completes in 60 s or less	Model Selection & Training	Background job design
MLE-NF-003	Pages load in 3 s or less for 20 concurrent users	API Gateway, Web UI	Layered stateless API
MLE-NF-004	Runs on Windows, Linux and macOS via Docker	Packaging	3.6 Docker
MLE-NF-005	Usable by a first-time user without training	Web UI	4.5 UX Design
MLE-NF-006	Failed jobs are contained (no partial model); saved data survives a restart	Model Selection & Training, Relational DB, Logging & Audit	3.7 R1; 4.4 Error Handling
MLE-NF-007	At least 20 datasets and 100 runs per user; lists paginated at 20 items	Data Workspace, Experiment Mgmt & Reporting, Relational DB	Pagination in list endpoints
MLE-SR-001	Passwords stored hashed	Auth & Access Control	SAD 3.9 Spoofing
MLE-SR-002	All traffic over HTTPS (TLS 1.2+)	Deployment / Gateway	SAD 3.9 Information disclosure
MLE-SR-003	Uploads validated for type, size and content	Input Validation & Security Controls	SAD 3.9 Tampering
MLE-SR-004	Only system-created model artifacts are loaded	Model Selection & Training	SAD 3.9 Tampering / EoP

Req ID	Requirement (short)	Architecture component	Design reference
MLE-SR-005	Per-user data isolation and audit log of actions	Auth, Logging & Audit	SAD 3.9 Repudiation, Info disclosure
MLE-SR-006	Request-size, rate, job-quota and run-time limits	API Gateway, Input Validation & Security Controls, Model Selection & Training	SAD 3.9 Denial of service
MLE-SR-007	Session expiry, login throttling, secure cookies and CSRF protection	Auth & Access Control	SAD 3.9 Spoofing, Tampering
MLE-SO-001	Confidentiality of datasets, models and results	Auth & Access Control, File Store, API Gateway	SAD 3.9 Information disclosure
MLE-SO-002	Integrity of uploaded data, models and metrics	Input Validation & Security Controls, Model Selection & Training	SAD 3.9 Tampering
MLE-SO-003	Availability against abusive or oversized requests	Input Validation & Security Controls, API Gateway, Model Selection & Training	SAD 3.9 Denial of service

3.9 Security Architecture

Security objectives (identical to SRS section 5.1.1: MLE-SO-001 Confidentiality, MLE-SO-002 Integrity, MLE-SO-003 Availability): (1) Confidentiality of each user's datasets, models and results. (2) Integrity of uploaded data, trained models and reported metrics. (3) Availability of the service against abusive or oversized requests. These are delivered under the Core Platform & Security epic (WBMME-11).

Threat modelling with STRIDE:

Threat	ML Evaluator scenario	Mitigation (component)
Spoofing	Attacker logs in as another user or reuses a stolen session	Hashed passwords (PBKDF2/bcrypt), 30 minute session expiry, login throttling, secure and HttpOnly cookies (Auth). Requirements: MLE-SR-001, MLE-SR-007
Tampering	Malicious CSV or modified model file changes results or runs code	File type, size and content validation; store artifacts outside web root; load only system-generated artifacts; no unpickling of user files; CSRF tokens on state-changing requests (Input Validation, Model Training). Requirements: MLE-SR-003, MLE-SR-004, MLE-SR-007
Repudiation	User denies deleting a dataset or an admin denies a change	Audit log of logins, uploads, trainings, deletions and admin actions with user ID and timestamp (Logging & Audit). Requirement: MLE-SR-005(b)
Information disclosure	One user reads another user's datasets or runs; sensitive data in logs or errors	Ownership check on every query and file path; HTTPS (TLS 1.2+); generic error messages; no raw data in logs (API, Auth, Logging). Requirements: MLE-SR-002, MLE-SR-005(a)
Denial of service	Huge uploads or many concurrent training jobs exhaust the server	Upload size limit, row/column limits, rate limiting, per-user job quota, training timeout (API, Input Validation). Requirement: MLE-SR-006
Elevation of privilege	Student reaches admin pages or APIs	Role-based access control on every view and endpoint; admin routes separated; deny by default (Auth). Requirements: MLE-F-002, MLE-F-016, MLE-SR-004

Trust boundaries: (a) browser to server (untrusted input, HTTPS required); (b) API layer to ML services (all data already validated); (c) application to file store and database (access only through the data layer).

4. Design

4.1 Design Overview

The design follows the layered architecture chosen in 3.5. A request travels Web UI, API Gateway, a service component (Data Workspace, Model Selection & Training, Evaluation, Experiment Management), and then the Relational DB and File Store. The API Gateway applies authentication, authorisation, CSRF checks, rate limiting and request-size limits before any service runs, so no service has to repeat them.

Main design decisions:

- Every dataset, run and result row carries an owner (user ID); every query and file path is filtered by that owner (MLE-SR-005).
- Training runs as a background job. The API returns a run ID immediately and the UI polls the run status; a job that fails or exceeds 300 s leaves no partial model (MLE-NF-006).
- Preprocessing (missing values, encoding, scaling) is fitted on the training split only, to avoid data leakage.
- Models are stored only as files created by the system under the owner's directory, and are loaded only from there (MLE-SR-004).
- Model types and hyperparameter ranges come from one registry (SRS Table 4.3-1), so the UI form, the validator and the trainer use the same source.

Main data entities and their relationships are shown in Figure 2.

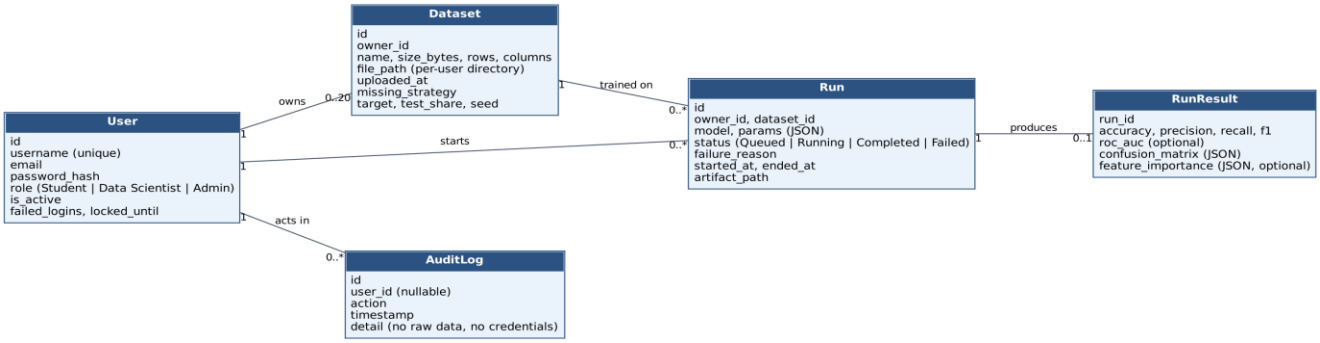


Figure 2: Class diagram of the main data entities (User, Dataset, Run, RunResult, AuditLog).

Notes on the entities: a User owns many Datasets and Runs. A Run refers to exactly one Dataset and has at most one RunResult (absent while queued, running or failed). AuditLog entries are written by the Logging & Audit component and are never edited by users.

4.2 UML Sequence Diagrams

Three key interactions are shown: login (SD-1), dataset upload (SD-2) and training with metrics (SD-3). Participants are the components of the component diagram in 3.3.

Figure 3: Sequence diagram SD-1, Log in (MLE-F-001, MLE-SR-001, MLE-SR-005, MLE-SR-007)

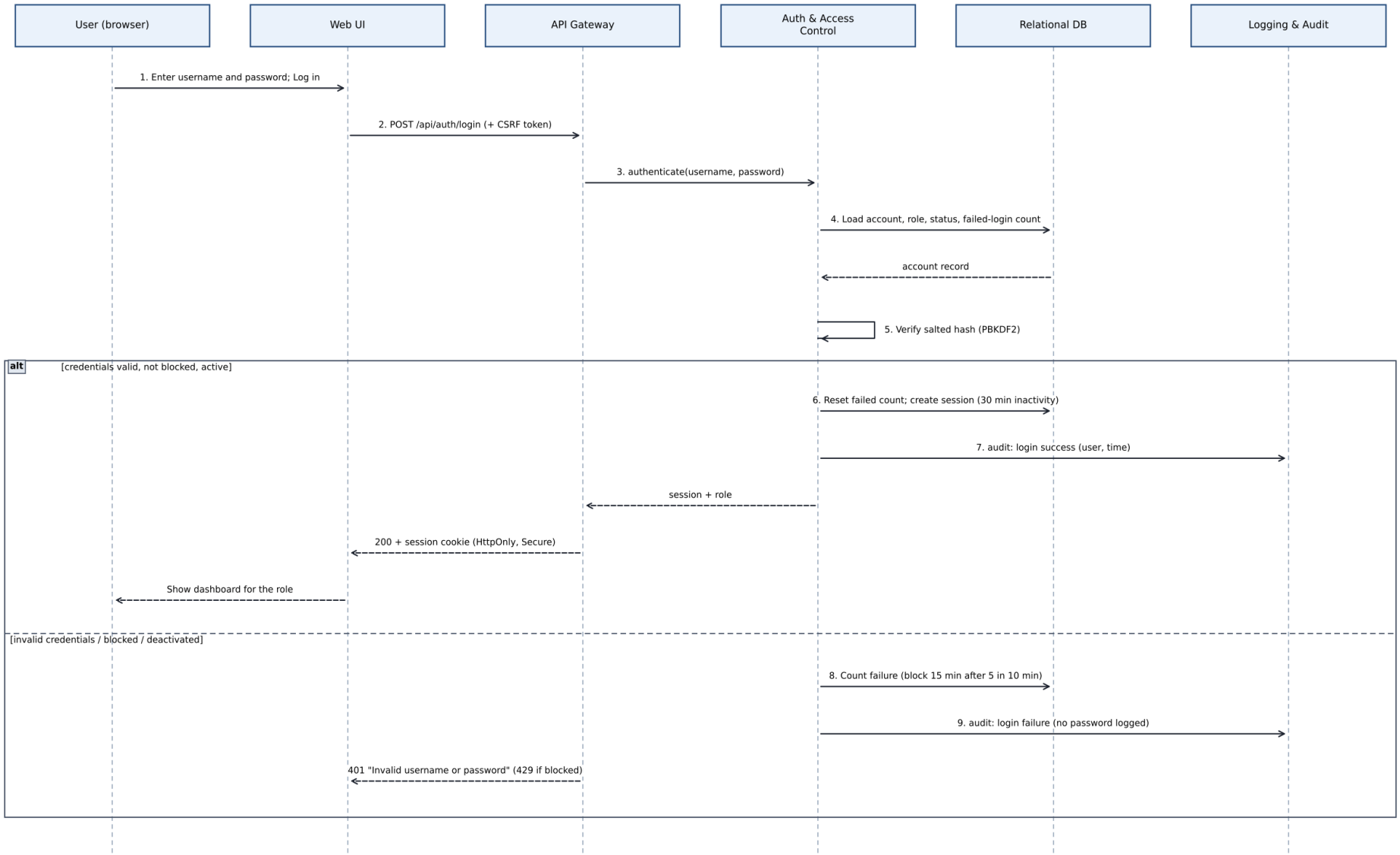


Figure 3: Sequence diagram SD-1, Log in.

Figure 4: Sequence diagram SD-2, Upload a dataset (MLE-F-003, MLE-F-004, MLE-SR-003, MLE-SR-006)

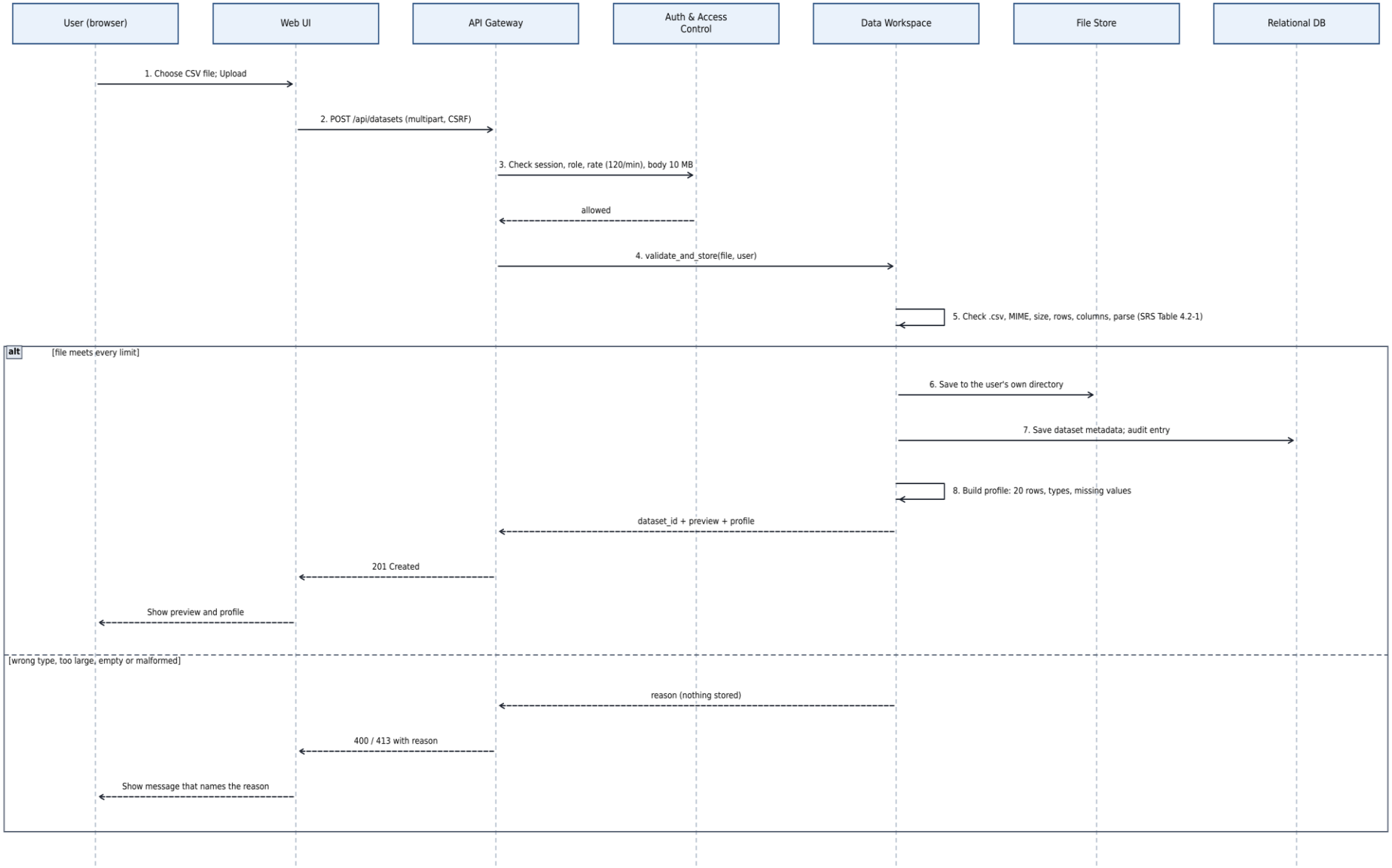


Figure 4: Sequence diagram SD-2, Upload a dataset.

Figure 5: Sequence diagram SD-3, Train a model and view metrics (MLE-F-007 to F-011, MLE-F-013, MLE-SR-006, MLE-NF-006)

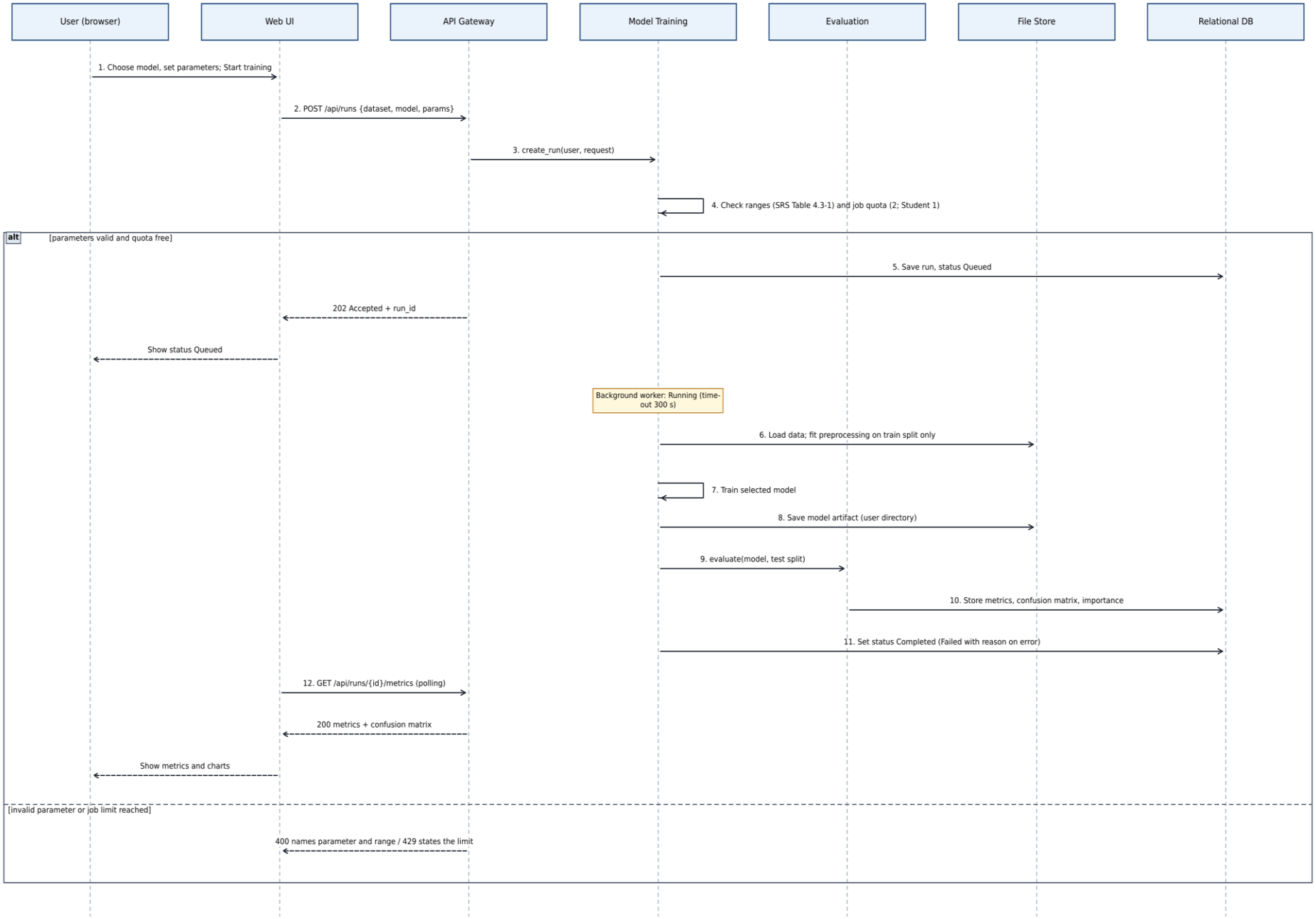


Figure 5: Sequence diagram SD-3, Train a model and view metrics.

4.3 API Design

All endpoints return JSON, require a valid session (except register and login), and require a CSRF token for POST, PATCH and DELETE. Common failure codes are described in 4.4. Lists are paginated at 20 items per page (MLE-NF-007).

Component 1: Auth & Access Control

Method	Endpoint	Request	Response / errors
POST	/api/auth/register	username, email, password, role (Student or Data Scientist)	201 with user ID; 400 on weak password or duplicate user.
POST	/api/auth/login	username, password	200 and session cookie (Secure, HttpOnly, 30 min); 401 on wrong credentials; blocked for 15 min after 5 failures in 10 min.
POST	/api/auth/logout	none	204; session destroyed.
GET	/api/admin/users	page (Admin only)	200 list of users; 403 for other roles.
PATCH	/api/admin/users/{id}	role or active flag (Admin only)	200 updated user; 403 for other roles; change written to the audit log.

Component 2: Data Workspace

Method	Endpoint	Request	Response / errors
POST	/api/datasets	multipart CSV file (at most 10 MB)	201 with dataset ID, 20-row preview and profile (types, missing values); 400 for wrong type, empty or malformed file or limits exceeded (20 to 50,000 rows, 2 to 100 columns); 413 if body over 10 MB.
GET	/api/datasets	page	200 list of the caller's datasets only.
GET	/api/datasets/{id}	none	200 metadata, preview and profile; 404 if not found or not owned by the caller.
DELETE	/api/datasets/{id}	none	204; removes file and metadata; audit entry written.

Component 3: Model Selection & Training, Evaluation and Experiment Management

Method	Endpoint	Request	Response / errors
GET	/api/models	none	200 catalogue of the four classifiers with hyperparameter names, defaults and allowed ranges.
POST	/api/runs	dataset ID, target column, model, hyperparameters, test split, missing-value and encoding options	202 with run ID and status queued; 400 for out-of-range values or target with fewer than 2 or more than 20 classes; 429 if the active job limit is reached.
GET	/api/runs	page	200 run history of the caller (dataset, model, parameters, status).
GET	/api/runs/{id}	none	200 status (queued, running, completed, failed, timed out) and parameters; 404 if not owned.

Method	Endpoint	Request	Response / errors
GET	/api/runs/{id}/metrics	none	200 accuracy, precision, recall, F1, ROC-AUC where available, confusion matrix and feature importance where available; 409 while the run is not completed.
GET	/api/runs/compare	ids (2 or more run IDs)	200 side-by-side metrics and parameters.
GET	/api/runs/{id}/export	none	200 CSV file; cells beginning with = + - or @ are neutralised.

Interface rules: the endpoint names above are the contract between the Web UI and the API; they are fixed at the start of Sprint 1 so that the front end and back end can be built in parallel.

4.4 Error Handling, Logging and Monitoring

Error handling

Code	Situation	Behaviour
400	Invalid input (file type, size limits, hyperparameters, malformed CSV)	Nothing is stored; the response names the field and the rule that failed, with no stack trace.
401	Not logged in or session expired	UI redirects to the login page; entered work is not lost on the server.
403	Role not allowed or CSRF token missing or invalid	Request rejected and recorded in the audit log.
404	Resource does not exist or belongs to another user	Same message for both cases so existence is not revealed.
409	Metrics requested before the run completes	UI keeps polling and shows the current status.
413	Request body over 10 MB	Rejected before the file is parsed.
429	More than 120 requests per minute, or active job limit reached	Response states when to retry.
500	Unexpected error	Generic message with a correlation ID; details only in the server log.

Training failures: if a job raises an error or exceeds 300 s, the run is marked failed or timed out, the partial model file is deleted, and the user sees a short reason. Data already saved (datasets, finished runs) is unaffected and survives a restart (MLE-NF-006).

Logging

- Structured application log (timestamp, level, user ID, request ID, component, message) for requests, training jobs and failures.
- Audit log in the database for logins, failed logins, uploads, deletions, training starts and admin changes (MLE-SR-005).
- Never logged: passwords, session tokens, and raw dataset values.

Monitoring

- A health endpoint (GET /health) returns 200 when the app and database respond; used by Docker and the demo.

- CI reports test results and coverage on every pull request; failed jobs and error counts are checked in the application log during testing.

4.5 UX Design

Design principles: one task per screen, a visible progress path, plain language, and error messages that say what to fix (MLE-NF-005: usable without training).

Screen	Content and behaviour
Register / Login	Short forms with inline validation; lockout message shows how long to wait.
Dashboard	Lists the user's datasets and recent runs, with buttons for Upload and New run. Admin sees a link to the admin console.
Upload	File picker with the rules (CSV, up to 10 MB) shown beside it; after upload shows a 20-row preview and column profile.
Prepare data	Choose the target column, missing-value and encoding options and the test split with defaults pre-filled.
Choose model	Four model cards; selecting one shows hyperparameters with defaults and allowed ranges; invalid values are flagged before submission.
Run status	Shows queued, running, completed, failed or timed out and refreshes automatically; Cancel is not offered in the committed scope.
Results	Metric cards, confusion matrix heat map, feature importance bar chart (Chart.js), and an Export CSV button.
History and Compare	Paginated run table; select two or more runs to open a side-by-side comparison.
Admin console	Users with role and active toggles; dataset overview; audit log view.

Flow: Login, Dashboard, Upload, Prepare data, Choose model, Run status, Results, then History or Compare. Accessibility basics: labels on all inputs, keyboard navigation, and chart values also shown in a table so colour is not the only cue.

4.6 Open Issues and Next Steps

#	Open issue	Next step
1	Exact API field names and JSON shapes	Freeze in a shared API sheet at the start of Sprint 1; update 4.3 if they change.
2	Background worker choice (thread or process pool versus Celery)	Start with a Python worker; revisit only if the 300 s time-out cannot be enforced reliably.
3	Whether PostgreSQL is used in the demo or SQLite only	Decide with the team during Docker work (Sprint 2); no code depends on it.
4	Optional features (SHAP, PDF report, Celery)	Not committed in the SRS; schedule only if time remains after Sprint 2 stories.
5	Requirement changes from team review	Update 3.8 and 3.9 together with the SRS so IDs stay consistent.
6	Test cases for MLE-NF-006, NF-007, SR-006 and SR-007	Ask the Test Plan owner to add them to Test Plan section 3.

5. Appendices

5.1 Glossary

Term	Meaning
ADR	Architecture Decision Record: a short note recording an architecture decision and its reason.
API	Application Programming Interface: the set of endpoints the Web UI calls.
CI/CD	Continuous Integration and Continuous Delivery: automatic build and test on every push (GitHub Actions).
CSRF	Cross-Site Request Forgery: an attack that makes a logged-in browser send an unwanted request.
CSV	Comma-Separated Values file, the only upload format.
MVT	Model-View-Template, the structure of a Django web application.
ML	Machine learning.
NF	Non-functional requirement (MLE-NF-###).
RBAC	Role-Based Access Control: permissions depend on the user's role (Student, Data Scientist, Admin).
RTM	Requirements Traceability Matrix: maps requirements to design and tests.
SAD	Software Architecture Document (this document).
SO / SR	Security objective (MLE-SO-###) / security requirement (MLE-SR-###).
SRS	Software Requirements Specification.
STP	Software Test Plan.
STRIDE	Threat categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
TLS	Transport Layer Security, used for HTTPS.

5.2 References

- ISO/IEC/IEEE 42010: Systems and software engineering, Architecture description.
- OWASP Top 10 and OWASP Application Security Verification Standard (ASVS).
- NIST SP 800-160: Systems Security Engineering.
- WBMME Software Requirements Specification, Parts A and B, and Software Test Plan.
- Django 5 documentation (security, authentication, CSRF) and scikit-learn documentation (classifiers and metrics).
- PES University SE Mini-Project Guidelines 2026 and Deliverables Part-1 and Part-2.

5.3 Tools

Purpose	Tool
Diagrams	Graphviz (component and class diagrams); Python with matplotlib (sequence diagrams)
Implementation	Python 3.11, Django 5, pandas, scikit-learn, joblib, Chart.js
Version control and tracking	Git, GitHub (repository, issues, Project board, pull requests)
CI and quality	GitHub Actions, pytest, pytest-cov, flake8 or SonarQube
Testing	Selenium (UI), Postman (API), JMeter or Locust (performance), OWASP ZAP (security scan)
Packaging	Docker